REBEL LEADERS / FIELD TOOL 01



AI Stalemate

THE LEAD DEVELOPER FIELD GUIDE

Explain the concerns. Work out the next step with your PM.

Nic Payne

For you, the lead developer

OPEN YOUR LEAD DEVELOPER LAB >

START WITH A REAL SITUATION. DECIDE WHAT HAPPENS NEXT.

WORKING EDITION / REBEL-LEADER.COM

Your PM wants to try AI. You have questions.

You may see something useful in your PM’s idea. You can also see code to check, mistakes to fix, and software someone will have to maintain. Explain those concerns in terms your PM can work with.

What are you hoping to improve?

Ask your PM what they want to make better. Perhaps support staff copy repair requests into two systems and customers have to call again when one is missed. That gives you a specific problem to discuss.

What is your colleague worried about?

Show your PM the concern: the app must prevent one customer from seeing another’s records, and someone needs time to review the code. Say what you would need to check and who could help.

What have you assumed about each other?

‘They asked me to try it by Friday’ describes what happened. ‘They don’t care about quality’ adds a motive. You may have good reasons to be frustrated. You still need to ask.

What could you work out together?

You might agree to try something small. You might need permission, advice, or more information first. You might decide this use is not worth pursuing. All of those can be useful answers.

> The aim is to understand the opportunity and the concerns well enough to decide what to do next.

Give them something they can picture.

‘We need to use AI’ gives your colleague very little to work with. What would be better for someone if this went well? Start there.

Describe an ordinary day

Our support team copies every repair request into two systems. Sometimes they miss one, and a customer has to call again. We want to stop making customers chase work they have already requested.

Explain why you care

Saving time matters because people could use it for something else: helping a customer, investigating a difficult fault, or finishing work that keeps being postponed. Say what you hope that time would make possible.

Leave room for their answer

Ask what they would change about your idea. They may see a simpler way to solve the problem, or a part you have missed. If their answer cannot change the proposal, it is not much of an invitation.

Think beyond this one task

You might learn a way of working the team can trust and teach to others. That could give people more confidence and room to grow. Treat it as a possibility to explore together, not a reward you can promise.

> Try: ‘I can see why you want to fix this. I’m worried about who could see the customer records. Can we work through that and make time for someone to check the code before we try it?’

What would need to change?

A concern is easier to work with when you know what it refers to. Ask for an example. Sometimes you can change the plan. Sometimes the answer is no, and you need to understand why.

‘The code is not good enough.’

Which checks does it fail? Would a smaller task, a good example, or clearer instructions help? Keep the checks you normally require before accepting work.

‘Reviewing it takes longer.’

Ask where the extra work appears. Count the author’s checking, someone else’s review, and the fixes. A faster first draft can still take more work to finish.

‘We cannot put that data in the tool.’

Find out which data and which tool are involved. Ask the person responsible for permission. A test with made-up records may be possible, but check before starting.

‘It does not know our code.’

Ask what it is missing. A small task with a relevant example may be worth trying. If it needs access you cannot give it, change the task or stop.

‘We do not have time.’

Who would do the work and who would review it? What would they put off? Get that choice agreed. Quietly adding an experiment to someone’s evening is not a plan.

‘We tried this already.’

Ask what they tried and what happened. Use what they learned. Another experiment needs a reason: what is different, and what do you still need to find out?

GitHub: Copilot best practices

Thoughtworks: reducing AI development friction

> Try: ‘Can you show me where it goes wrong? I want to understand whether we can change the idea or should leave this use alone.’

Before you talk with your product manager.

Take five or ten minutes before the conversation. You do not need a perfect argument. You need to know what matters to you and what you still need to ask.

What happened?

Describe what was said or done, using words the other person would recognize.

What am I assuming?

Notice the motive you have assigned to them. Ask about it rather than presenting it as a fact.

Why does this matter to me?

Describe the checks, review, and maintenance you would take responsibility for. Include what could be useful about the idea as well as what worries you.

What have I actually seen?

Separate something you observed from something you expect. Both can matter, but they are different kinds of information.

What would change my mind?

Name a result or explanation that would make you reconsider. Leave room for a better idea.

What do I still need to ask?

Write the question you have not answered by yourself. That is a good place to start the conversation.

> Private notes stay only in this open tab. They are not saved or included in shared documents, and they disappear when you refresh or leave. You can also reflect without typing.

Ask what is behind your PM’s request.

Your PM may have heard something from customers or leadership that you have not. Ask what they want to improve and why it matters now. Then explain what you would need to do the work.

Who would this help?

Describe the people and the work. In our fictional example, support staff copy repair requests into two systems. An app might remove that repeated work and reduce missed requests.

Where did the number come from?

If someone says this could be worth $500,000 a week, ask how they worked it out. Is that money currently being lost, money they hope to earn, or time they expect to save? Say who supplied the estimate and who will check it.

What could the team learn?

People may become better at checking generated code or teaching others where it helps. Ask what would make the work worthwhile for them too. Do not promise promotion or rewards you cannot control.

What would it take to find out?

Ask what help, permission, training, or time is missing. Take that specific request to the person who can approve it. Name what other work would wait.

> Try: ‘Here’s what I’ve heard from the customer. I haven’t checked this figure yet. What else would you need to know before we decide whether to try it?’

Who should be able to see this?

Imagine a customer opening the app and seeing another customer’s repair history. Before connecting real systems, the people responsible need to agree who may see and change each kind of record—and verify that the software enforces it.

Decide who may do what

Name the users or groups, the records they may see, and whether they may change them. Decide who grants and removes access, including when someone changes jobs or leaves.

Check more than the sign-in screen

Signing in tells the system who someone is. It must also check what they are allowed to do. An API is a connection software uses to request data or actions; those requests need permission checks too. Hiding a button is not enough.

Limit what the AI can do

Say which code, data, and system it may use. Can it only read, or can it make changes, run commands, or release software? Agree who approves those actions and who can undo a mistake.

Ask the right people to verify it

The lead developer and the people responsible for security and data need to define and check the rules. Record their names, the checks, and where the results can be found. Filling in this form does not grant permission.

Microsoft: app roles and groups

Microsoft: verify API scopes and roles

OWASP: authorization checks

> If nobody has agreed or tested the permissions, resolve that first. A test using made-up records may help, but it also needs approval.

Can the team check the work it will be responsible for?

AI can produce code that looks plausible. Someone still needs to understand it, check it, and know what to do when it fails. Work out what help the team needs before giving it that responsibility.

Ask people to show their working

Can the developer explain the change, test who can access the data, inspect the code changes, and investigate a failure? Can they explain how to release it and restore the previous version? Find out by working through a small example together.

Make time for an experienced reviewer

Pair a developer with someone who can check this kind of work. Ask them to explain how they know the result is correct. Agree time for that conversation as part of the work.

Practise the whole change

Agree what the finished work must do. Keep a record of code changes, have someone review them, run the required tests, and practise releasing and undoing the change.

Keep the first task small

Choose a piece of work people can understand and check. Record what the team needs to learn before taking on more. Do not assume that writing more code means the team can safely maintain more software.

NIST: Secure Software Development Framework

GitHub: responsible use of Copilot agents

DORA: working in small batches

> ‘Prepare first’ is a useful answer when it names actual work: who will pair with whom, what they will practise, and what they need to demonstrate before starting.

What would be worth finding out?

Plan the experiment with your product manager. Explain which task is suitable, what needs checking, and how much review time you need. Ask them to help agree what other work will wait.

Say what you expect and why

For example: we think AI could help write permission tests faster because many of them are similar. That is a prediction. We have not shown it yet.

Choose the task and the comparison

Name exactly what you will try and what you will leave out. Choose similar work done without AI so the comparison is useful. Note differences in task size, experience, or required checks.

Check permission

Record the approved tool and version, the code and data it may use, and who confirmed permission. Permission for this experiment does not cover live customer systems or wider use.

Agree the time and checks

Include time to prepare, write, check, review, and fix the work. Decide what must pass before you accept it. Agree what other work will wait and who can approve that change.

Decide what would count as useful

Write your prediction separately from the smallest improvement that would make another use worthwhile. Agree when to stop—for example, if a permission check fails or you use up the review time.

Give the plan to the people doing it

Name the author, reviewer, person who can approve the result, and review date. Confirm they have agreed. If something is missing, pause and name who will resolve it.

> The idea of choosing how much time to invest draws on Ryan Singer’s Shape Up (basecamp.com/shapeup). An experiment that never started has no result.

Did the faster draft save any time?

Count your work and the reviewer’s work, including checks and fixes. Two people working for 30 minutes use 60 person-minutes. Give your PM the full picture of time and quality.

Work	Without AI	With AI
Preparation	10 min	15 min
Writing the first version	45 min	12 min
Author checking and testing	15 min	25 min
Someone else reviewing	15 min	30 min
Fixes	5 min	15 min
Later follow-up	0 min	0 min
Total working minutes	**90 min**	**97 min**

A separate fictional example

These numbers describe one small task writing software tests. The finished tests passed the required checks after fixes. This is separate from the preparation example in the Lab.

What happened?

AI cut drafting by 33 minutes. Checking, review, and fixes used 40 more. The whole task therefore took seven extra minutes. The reviewer spent 30 minutes, twice the agreed 15-minute limit; that limit was exceeded and needs to be addressed before another experiment.

What else do you need to know?

Record failed checks, later defects, tool costs, and anything that makes the tasks hard to compare. Ask the author and reviewer how the work felt. One task cannot tell you what will happen everywhere.

> Enter 0 when no time was spent. Leave a box blank when you do not know. A partial total cannot show a saving.

You do not have to make the experiment a success.

Look at what happened against what you agreed beforehand. The result may change your mind. That is part of the point.

CONTINUE

The result supports using AI for this task under the agreed conditions. Say where that permission ends and when you will check again.

REVISE

Something looks useful, but the task, instructions, or review needs to change. Describe the change before asking people to try again.

STOP

The result does not justify this use, or an agreed limit was broken. Explain why you are stopping and keep what you learned.

PAUSE

You are missing permission, information, time, or a person who needs to be involved. Name what is missing, who will resolve it, and when you will return. You can choose this before an experiment.

> Write what you decided, why, who will do what next, and when. If you have not agreed, keep the record marked as a proposal and say what still needs deciding.

The next step is preparation.

Fictional case: your PM wants to help support staff stop copying repair requests between two systems. You see questions about who can see the customer records and whether the team can check and release the code safely.

What the PM knows

The sales director says reducing the delay could create $500,000 of value a week. Nobody has checked the calculation. The PM will ask Finance what the figure includes before treating it as a fact.

What the lead needs to check

Staff should see only the customer records their jobs require. The team needs tests that prove forbidden access is blocked, including direct requests to the app’s API—the connection other software uses.

What they propose

Prepare first. The data owner will decide who may see which records. The engineering manager will pair developers with an experienced reviewer to practise checking, releasing, and undoing a change.

What they record

PAUSE this use for now. Check progress next Tuesday. No experiment or connection to live customer data is approved. A later test with made-up records would need its own permission.

What they tell leadership

‘We can see why this could help. Before we try it, we need to agree who can see the customer records and check that the team can release a change safely. Can we make time for that preparation? We’ll review it next Tuesday.’

> They have a proposal someone can answer, with specific work and people responsible. They have not measured an AI productivity gain.

Bring one real conversation.

The Guide explains the questions. The Lab gives you somewhere to answer them and prepare a document you can share. You can start where you are; you do not need to fill in every box.

1. Before you talk

Write private notes, or think through the questions without typing. What matters to you, what are you assuming, and what do you still need to ask?

2. Conversation

Share what you want to improve and ask about the concerns. Choose what to do next. Build Next Step Together, check it, then copy, download, or print it. Mark a proposal as a proposal.

3. Experiment, if useful

Agree the task, time, checks, permissions, people, and review date. Build and review the Experiment Plan before starting. You can skip this step if your next move does not involve an experiment.

4. Results

Return after the experiment. Enter all the time spent, the quality checks, and what happened. Build the Results Record. Leave unknown values blank.

5. Next step

Write the decision and the reason. Name who will do what next and when you will check again. Build the Decision Brief and review it before sharing.

Shared drafts are saved in this browser on this device. Download work you need to keep before changing devices or clearing browser data. Private notes are never saved. Load Examples offers fictional answers; if the step contains work, it asks before replacing it.